

Re-establishing Fetch-Directed Instruction Prefetching: An Industry Perspective

Yasuo Ishii¹, Jaekyu Lee², Krishnendra Nathella³, Dam Sunwoo⁴
Arm, Austin, TX, USA

{yasuo.ishii, jaekyu.lee, krishnendra.nathella, dam.sunwoo}@arm.com

Abstract—Instruction prefetching can play a pivotal role in improving the performance of workloads with large instruction footprints and frequent, costly frontend stalls. In particular, Fetch Directed Prefetching (FDP) is an effective technique to mitigate frontend stalls since it leverages existing branch prediction resources in a processor and incurs very little hardware overhead. Modern processors have been trending towards provisioning more frontend resources, which bodes well for FDP as it requires these resources to be effective. However, recent academic research has been using outdated and less than optimal frontend baselines that employ smaller structures, resulting in equivocal outcomes. This paper presents a detailed FDP microarchitecture and evaluates two improvements, better branch history management and post-fetch correction. Our mechanism provides a 41.0% speedup over the baseline (no prefetching, no FDP) with only 195 bytes of hardware overhead and outperforms the 1st Instruction Prefetching Championship (IPC-1) winners that had a 128KB storage budget. We believe that our FDP-based frontend design can serve as a new reference baseline for instruction prefetching research to bridge the gap between academia and industry.

Index Terms—Instruction prefetching; branch prediction;

I. INTRODUCTION

The instruction footprints of both data-center and client applications have been growing larger, causing more instruction cache (I-cache) misses and frequent pipeline stalls in modern processors. Instruction prefetching [1]–[18] is a well-known technique to reduce I-cache misses. Next line prefetching [19], [20] can effectively cover sequential I-cache accesses, but its effectiveness is drastically reduced for discontinuous control flows due to various branches. Temporal prefetching mechanisms [4]–[7], [18] try to tackle discontinuity by tracking temporally correlated streams of I-cache misses. If an instruction address that triggers a stream is seen again, subsequent addresses will be prefetched. However, temporal prefetching generally requires huge metadata storage to be effective.

Fetch-Directed Prefetching (FDP) [8]–[12], instead, leverages existing branch prediction components, such as direction predictors [21]–[23] and the Branch Target Buffer (BTB), that are highly accurate and well provisioned in modern processors, making FDP effective with marginal hardware overhead. FDP is built on a decoupled frontend design [24]. The instruction addresses predicted by the branch predictor are stored in a dedicated structure called a Fetch Target Queue

(FTQ). I-cache lines that FTQ entries point at are fetched into the I-cache before the demand request. The decoupled frontend enables the processor to run ahead and predict future instruction streams independent of instruction fetch. Recently disclosed commercial CPU designs [25]–[28] indicate that they employed a variant of FDP.

FDP relies on accurate branch predictors and sufficiently large BTBs to cover the control flow. In particular, the BTB is solely used to detect branch information during instruction fetching. FDP can run into some issues when the BTB does not well cover the predicted control flow. One issue stems from branch history management. The branch history is used to access branch predictors, so FDP needs to update the branch history from all branches regardless of their outcomes (taken or not taken). If the BTB does not cover some *not-taken* branches, the predictor cannot make a direction prediction, and the branch history is not updated. Another issue occurs when a *taken* branch misses in the BTB. A branch that misses in the BTB goes undetected even if the branch direction predictor could have predicted the branch outcome correctly, leading to the wrong, sequential control path. Thus, to cover more branches and improve the effectiveness of FDP, commercial processors have been trending towards adding more resources in the branch predictor, as seen in recent CPUs [25]–[29].

Academic research, however, shows the opposite trend. First, a less-than-optimal frontend baseline is often used. In the recently held 1st Instruction Prefetching Championship (IPC-1) [14], although the baseline has a decoupled frontend and basic FDP capability [8], its shallow 12-instruction FTQ limited the runahead capability of FDP. To understand the impact of the limited FDP, we compared various instruction prefetching mechanisms, including a next-line, the top-3 prefetchers, D-JOLT [30], FNL+MMA [31], and EIP [18], from IPC-1, and perfect [32] with and without FDP. In this experiment, all mechanisms use the same framework with perfect target prediction used in IPC-1. Fig. 1 shows the speedup over the baseline (no prefetching, no FDP). As presented in IPC-1, the top-3 prefetchers offer over 28% speedup, achieving close to perfect prefetching (30.6%). The simplistic FDP only with a larger 192-instruction FTQ shows a 30.2% speedup. The top-3 IPC-1 prefetchers with FDP provide little benefit over FDP alone. These results show that 1) the baseline used in academia lags behind that of industry and 2) employing additional prefetching mechanisms on top of FDP cannot be justified without reasonably high performance benefits.

This paper is an extension of the following paper: Y. Ishii, J. Lee, K. Nathella and D. Sunwoo, “Rebasing Instruction Prefetching: An Industry Perspective” in IEEE Computer Architecture Letters, vol. 19, no. 2, pp. 147–150, 1 July-Dec. 2020.

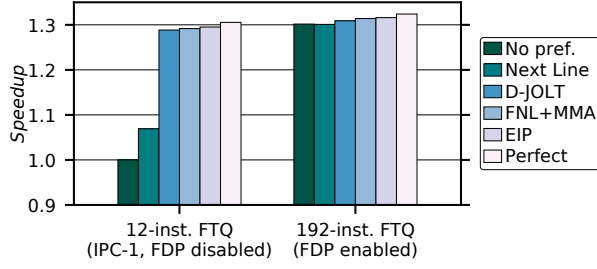


Fig. 1. Prefetching limit study using the IPC-1 framework.

TABLE I
BTB CAPACITY GAP BETWEEN ACADEMIA AND INDUSTRY (# ENTRIES).

Academia	BTB	Industry	BTB
Shotgun [12]	2.1K	AMD Zen2 [29]	7K
Confluence [10]	1.5K	Samsung Exynos M3 [27]	16K
Divide&Conquer [13]	2K	Arm Neoverse N1 [26]	6K

Second, as shown in Table I, many academic papers used a smaller BTB for their evaluations, while commercial processors, based on recently disclosed information, implemented a larger BTB. Instead, BTB prefetching mechanisms [10]–[13] are proposed to compensate for the limited BTB capacities. Upon an I-cache miss and subsequent fill, BTB prefetching mechanisms pre-decode instructions in the same I-cache line and add identified branch information (target) in the BTB. These mechanisms often require more complex basic-block-based BTB structures. Moreover, they can cause BTB pollution since branch target information will be blindly inserted even for strongly biased not-taken branches.

To reduce the gap between frontend baselines in industry and academia, we revisit an FDP design and provide comprehensive microarchitecture details, which is more representative of a modern CPU. We also evaluate two features to further improve FDP. First, using *taken-only* branch *target* history, as opposed to branch *direction* history from *all* branches used in most academic papers, can make FDP more tolerant of BTB-miss not-taken branches. Branch direction history can be easily disturbed by BTB-miss not-taken branches. Using taken-only target history naturally resolves this issue and allows BTBs to keep taken branches only, improving BTB utilization.

Second, post-fetch correction (PFC) can quickly recover the misprediction penalty from BTB-miss taken branches by eagerly re-steering the prefetched instruction stream. Even if a branch misses in the BTB, its direction can be accurately predicted by the direction predictor. Also, the branch target can be obtained shortly after the BTB miss for a PC-relative branch (offset embedded in an instruction) or a function return (target obtained from Return Address Stack (RAS)). Without PFC, upon BTB-miss taken branches, a core will continue to execute wrong-path instructions until the pipeline flush that is initiated after the branch mispredictions are identified. PFC was previously used only for simple unconditional and loop branches [25], and we extend PFC for all PC-relative

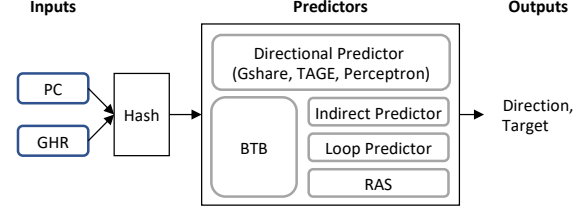


Fig. 2. The overview of branch prediction.

conditional branches.

This paper claims the following key contributions:

- We present a comprehensive microarchitecture for FDP design and evaluate two features that improve FDP.
- Our FDP design performs 41.0% better than the baseline (no prefetching, no FDP) with only 195 bytes of hardware overhead and outperforms the IPC-1 winners that had a 128KB storage budget.
- We find that adding a dedicated prefetcher on top of FDP provides only marginal performance improvement for our evaluated workloads.
- Our improved FDP design will bridge the gap in frontend designs between industry and academia, and it can be used as the baseline for future instruction prefetching research.

II. BACKGROUND

A. Branch Prediction

Branch prediction mechanisms are crucial for sustaining core performance by continuously supplying instructions to the pipeline, which has spurred a wide range of studies on branch prediction. Modern processors include various branch predictors that help with branch directions, targets, loops, and return addresses. Direction predictors [21]–[23], [33] predict the direction (taken or not taken) of branches. Each predictor entry has a saturating counter, which is trained using branch outcomes, and its most significant bit (MSB) indicates the predicted direction. The BTB stores branch targets and is used for target prediction. Dedicated indirect branch predictors [34] can be used to predict indirect branches that may have multiple targets. Function return addresses are pushed into the RAS upon function calls, and the address at the top of the RAS is used for return instructions. Loop predictors also exist to identify loops with their loop iteration counts. Fig. 2 shows an overview of branch prediction with these components.

$$direction_history = (direction_history \ll 1) \mid branch_taken \quad (1)$$

$$target_hash = (instruction_address \gg 2) \oplus (target \gg 3) \quad (2)$$

$$target_history = (target_history \ll 2) \oplus target_hash \quad (3)$$

Maintaining precise recent branch histories is essential for accurate predictions since the history is used to index branch predictor entries. Incorrect history leads to accessing a wrong entry for the prediction, potentially leading to a branch misprediction. Branch outcomes (taken or not taken) are collected after instructions are fetched [35] and maintained in the global

history register (GHR). The GHR can be updated using either directions from *all* branches (e.g., Eq. 1) or targets from *taken* branches (e.g., Eq. 3).

Branch predictors can be accessed at the fetch stage to improve prediction latency. Although the instruction type is not identified yet, the BTB may store the fetch address from the previous instance of a branch instruction. Upon a BTB hit with a predicted taken direction, the instruction fetch will be re-directed to the predicted address. As described in Section I, commercial processors are trending towards increasing the size of branch predictors. Predictors are often implemented using multiple ports or multiple banks with cost-effective single-ported RAM [35] to sustain the latency and bandwidth for such large structures. Also, similar to the multi-level cache hierarchy, the multi-level BTB hierarchy can be implemented [25]–[28].

B. Fetch Directed Prefetch

FDP [8] was proposed to exploit the existing branch prediction mechanism to initiate instruction prefetches. The branch predictor is decoupled from the instruction fetch logic that consists of the instruction translation lookaside buffer (I-TLB) and the I-cache. The branch predictor continues to predict future fetch addresses and pushes them into the FTQ. The BTB is accessed for addresses stored in the FTQ to produce the next fetch addresses without accessing the I-cache. During control flow prediction, if a series of branches are covered by the BTB, i.e., BTB hits, with the directions supplied from the direction predictors, FDP can effectively prefetch instruction streams. The GHR is updated based on branch predictor outcomes.

FDP enables a cost-efficient, high-performance frontend thanks to the reuse of existing branch prediction resources. However, its performance can be limited by the BTB capacity. When taken branches miss in the BTB, these branches are not detected and cause the control flow to continue on the sequential path, leading to wrong-path instruction fetching. Moreover, the branch predictor will fail to update the GHR for all BTB-miss not-taken branches, leading to imprecise histories and further mispredictions.

III. IMPROVING FETCH DIRECTED PREFETCHING

In this section, we elaborate on how FDP suffers from issues previously discussed and how to overcome these issues with two enhancements, taken-only branch target history and post-fetch correction.

A. Taken-only Branch Target History

The direction history is widely used in academic research, but the run-ahead capability enabled by FDP can easily disturb precise direction history management. FDP relies on the BTB to collect branch history information. With this approach, BTB-miss *not-taken* branches go undetected, and the branch predictor fails to push not-taken branch outcome into the GHR, incurring imprecise branch history. On the other hand, BTB-miss *taken* branches do not cause the same issue because the pipeline eventually gets an opportunity to fix branch

TABLE II
HANDLING BTB-MISS NOT-TAKEN BRANCHES.

History type	GHR fixup	Branch mis-predictions	Frontend stalls	BTB allocation
Target	No need	Fewer	None	Taken
Direction (no fix)	No	Most	None	All
Direction (fix)	Yes	More	More	All

history upon pipeline flush caused by branch misprediction. During the pipeline flush, the branch history information is unrolled, and the GHR can be updated based on taken branch information that caused the pipeline flush.

BTB-miss *not-taken* branches can be handled in two ways. First, we can skip the GHR update for these branches, but this may lead to more branch mispredictions because of imprecise history. Second, the GHR can be corrected once the branch has been identified after prediction. This scheme incurs more frontend flushes and backend pipeline stalls because the GHR correction may change the branch predictor output. Basic-block-based BTB constructions [11], [12] were proposed to eliminate these undetected (BTB-miss) not-taken branches. By keeping all branches in the BTB (exactly one branch per BTB entry by the definition of a basic-block), they can recognize BTB-miss not-taken branches upon prediction. However, this strategy can cause BTB pollution. For example, these schemes require dedicated BTB entries even for basic blocks with (almost) always not-taken branches (e.g., conditional branch for exception condition detection), which are less useful. Also, because of the variable size of basic-blocks, the BTB structure becomes more complex, including BTB indexing and additional fields to specify the basic-block start address, its size, and the branch offset.

Taken-only branch target history naturally resolves the imprecise branch history issue since not-taken branches are ignored for the GHR update. This strategy also allows the BTB to store only taken branches. These benefits led many commercial CPU designs to employ the target branch history [25], [36], [37], and we adopt it for our FDP design as well. Table II summarizes how BTB-miss not-taken branches can be handled and also shows the benefit of using target history. In Section VI-C, we compare the different history management policies to see how they affect performance and branch prediction accuracy.

B. Post-Fetch Correction

In the original FDP implementation, on a BTB miss, a branch goes undetected, and no prediction is made. This would lead to fetching wrong I-cache lines for taken branches until the branch misprediction is detected. Assuming that an undetected branch is not-taken is a valid approach, and some commercial processors behave this way [29]. However, this works well only if the instruction footprint fits in the BTB. Otherwise, FDP will frequently go down the sequential fetch flow, causing more branch mispredictions.

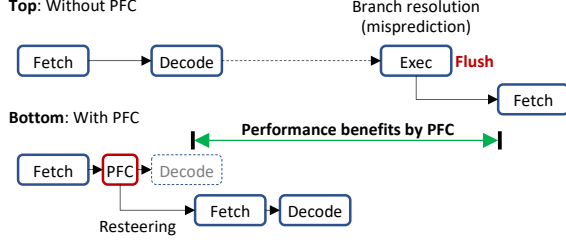


Fig. 3. The performance benefit of PFC.

However, we have observed that modern branch predictors [23] can correctly predict the direction of these BTB-miss branches. Based on this observation, we propose to add a post-fetch correction (PFC) capability to FDP. PFC was restricted to simple unconditional and loop branches [25], and we extend PFC capability for all PC-relative conditional branches. The idea is to unconditionally make direction predictions on all instructions (including non-branch instructions), as in the Alpha EV8 processor [35], and store the prediction information (direction for BTB miss branches) in the FTQ. When instructions are (pre-)decoded after the fetch stage, we can accurately identify all branch instructions. Branch targets can also be identified for PC-relative branches (offset embedded in an instruction) and function returns (target obtained from RAS). The FTQ is then checked whether 1) an unconditional branch instruction was predicted not taken or missed in the BTB, or 2) a conditional branch was predicted taken and missed in the BTB. If either condition is met, the FTQ is flushed, the GHR is fixed, and the prefetch stream resumes from the new branch target immediately. Otherwise, I-cache lines will be fetched from the sequential path.

Fig. 3 illustrates the performance benefit by PFC. Without PFC, the core continues to execute wrong-path instructions until the pipeline flush occurs after the execution stage. Instead, by eagerly detecting BTB-miss taken branches and fixing the control flow, PFC can reduce the branch misprediction penalty and prevent I-cache pollution, improving performance.

IV. COMPREHENSIVE FRONTEND DESIGN FOR FDP

We describe a comprehensive frontend design for FDP in this section. Fig. 4 shows an overview of our frontend design. Similar to existing FDP designs, the frontend contains separate branch prediction and instruction fetch pipelines, and the FTQ connects the two pipelines. We assume fixed-length 32-bit instructions in this paper, but FDP can handle variable instruction lengths as well.

A. FTQ Structure and Hardware Cost

As described in Section II-B, the FTQ is the only additional structure required by the FDP design. Each FTQ entry covers a 32-byte aligned instruction block so that all instructions in an FTQ entry fall into the same I-cache line. Each FTQ entry has the following fields:

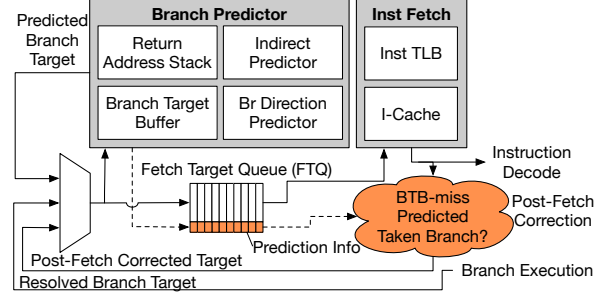


Fig. 4. Overview of frontend microarchitecture design.

TABLE III
HARDWARE OVERHEAD.

Field	Size
Start address	48-bit
Block predicted taken	1-bit
Block termination offset	3-bit
I-cache way	3-bit
State	2-bit
Direction hint	8-bit
Total (24-entry)	195 bytes

- *Instruction start address* (48-bit): The FTQ needs not store physical addresses since the I-cache tag lookup follows immediately after I-TLB lookup.
- *Predicted taken* (1-bit) to indicate whether the block is terminated by a predicted taken branch.
- *Block termination offset* (3-bit) by the taken branch within a 32-byte block (8 instructions).
- *I-cache way* (3-bit) to fetch I-cache line without extra I-cache tag access (§IV-C).
- *State* (2-bit) to track I-TLB/I-cache tag lookup completion: 0: Invalid, 1: Branch prediction completed and ready for address translation, 2: Address translation completed and is waiting for the I-cache fill, and 3: Ready to send instructions to the decode queue.
- *Branch direction hint* (8-bit): 1-bit per each instruction. This is only additional field to support PFC from the original FDP implementation.

Table III summarizes all fields of an FTQ entry and the total hardware overhead. We use a 24-entry FTQ based on an empirical study (§VI-G), so the total storage overhead is 195 bytes. Compared to a conventional FDP implementation, our proposed FDP incurs only 24 bytes (*branch direction hint*) hardware overhead.

B. Branch Prediction Pipeline

Branch predictors are accessed to predict the next instruction address. We model the high-bandwidth branch predictors similar to Alpha EV8 [35] as follows. First, we use 16-bank TAGE predictor tables, where each bank is a single-ported RAM. Since we can use the same branch history for all predictions in a given cycle thanks to taken-only branch

target history (§III-A),¹ TAGE can produce up to 16 direction predictions. Second, we use a 16B-indexed BTB, i.e., all branch instructions in the same 16B block will reside in the same BTB set. Our 4-bank, 4-way L2 BTB can read out up to 16 instructions simultaneously. Lastly, we assume that the branch predictor can produce up to 1 taken prediction per cycle to simplify the branch prediction design.

To utilize this bandwidth, we use a sequential instruction block to access branch predictors without bank conflicts. For example, with the instruction address A under our baseline 12-instruction per cycle prediction bandwidth (§V), 12 instructions (of which addresses range from A to $A+44$) will access the predictors concurrently. If a block has any predicted-taken branch with a BTB hit, the corresponding branch target from the BTB or indirect predictor will be fed into the prediction pipeline in the next cycle. As discussed in Section III-A, GHR will be updated by this predicted-taken branch. Otherwise, the prediction pipe continues on the sequential path (i.e., $A+48$) without updating the GHR. After the prediction, the 32-byte aligned sequential instruction block (up to 8-instruction) with its prediction information (direction hint and taken branch offset, if any) will be inserted into the FTQ.

C. Instruction Fetch Pipeline

The instruction fetch pipeline serves two primary purposes: I-cache fills and instruction fetches. First, we access the I-cache tag array to see whether the I-cache line associated with an FTQ entry exists in the I-cache. In particular, the two oldest entries that are ready (FTQ State: 1, §IV-A), will be selected to access the I-TLB, and the I-cache tag array will be subsequently accessed after the translation. If the I-cache tag access results in a cache miss, the state is updated (FTQ State: 2), and an I-cache fill request is initiated to the lower-level cache without waiting for the entry to be the head of FTQ. Upon a cache fill, the filled cache way information is written into *I-cache way* of the corresponding FTQ entry, and the state is updated accordingly (FTQ State: 3). Similarly, in case of a hit, the hitting way information will be saved in the FTQ entry, and the state is updated (FTQ State: 3).

Second, for instructions at the head of the FTQ, the corresponding I-cache line in the specified *I-cache way* will be fetched from the I-cache data array.² Then, instructions will be added into the decode queue. By decoupling I-cache fills and instruction fetches, we can initiate I-cache fills early regardless of any back-pressure from the decode queue and backend. An FTQ entry is released after all instructions from the FTQ entry are sent to the decode queue.

By pre-decoding instructions that are brought in from the I-cache data array, the instruction fetch pipeline can detect PFC candidate branches. Fig. 5 shows an example with an FTQ entry with the start address at offset 2 and the end of the block

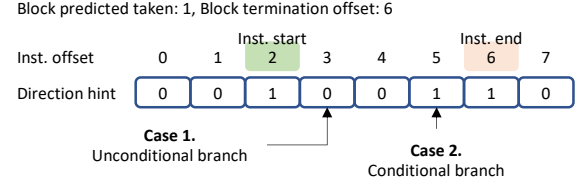


Fig. 5. PFC candidate branches from an FTQ entry.

at offset 6 by a predicted taken branch. PFC will be initiated for two different cases. The first case (Case 1) is when the fetch unit finds an unconditional branch whose offset within the block is less than the end offset ($3 < 6$ in the example). If the prediction hint for this instruction is 0, this was due to a wrong direction prediction. Otherwise, it was due to a BTB miss. The second case (Case 2) is by a PC-relative conditional branch whose offset is again less than the end offset with its direction hint set. This case is always caused by a BTB miss.

V. EVALUATION METHODOLOGY

We evaluate our proposed FDP using Champsim [38], an open-source trace-based simulator used in IPC-1. We run 50M instructions with 50M warmup using all workloads, including server, client, and SPEC [39], published by IPC-1 [14]. Each workload shows over 5% performance uplift with a perfect I-cache over a 32KB I-cache. We implemented the proposed comprehensive frontend microarchitecture with various branch predictors, including the TAGE [23], BTB, indirect [34], and RAS predictors.

We use core parameters similar to Intel Sunny Cove [40]. The branch predictors are sized based on recently announced commercial processors [26], [27], [29] and tuned for evaluated workloads. We use a 260-bit branch history length for both TAGE and ITTAGE. The branch history is updated only by taken-only branch targets, as shown in Eq. 3. The branch predictor bandwidth is double the fetch bandwidth to support the run-ahead capability [26]. We evaluated 2-entry (16-instruction) FTQ to disable FDP because this configuration limits the run-ahead capability. We evaluated and compared the following mechanisms using common parameters shown in Table IV:

- Baseline: no FDP, no prefetching.
- Next line (NL1): prefetch the next line upon a cache miss.
- The top-3 prefetchers from IPC-1: FNL+MMA [31], D-JOLT [30], EIP-128KB (with the original 128KB 34-way Entangled Tables) [18], and EIP-27KB (with a realistic 27KB 8-way Entangled Table).
- Perfect prefetching (Perfect): proposed in [32], where we assume that a prefetch brings the data into the cache instantaneously but still sends out the request to the memory subsystem to simulate the traffic overhead.
- FDP: our improved FDP with 24-entry (192-instruction) FTQ. PFC is enabled. FDP can be employed independent of other configurations.

¹Until a taken branch, all instructions can use the same target history.

²The cache line will be protected, i.e., not evicted, until the instruction fetch. Although this can be handled differently (e.g., I-cache fills check and flush the FTQ if a matching entry exists), we empirically confirm that this case rarely occurs (none from our simulations) and will not be discussed further.

TABLE IV
SIMULATION PARAMETERS.

Parameter	Value
CPU frequency	4GHz
Fetch Width	6-wide, 24-entry Fetch Target Queue
Decode Width	6-wide, 60-entry Decode Queue
Out-of-order Core	352-entry Re-order Buffer, 128-entry Unified Reservation Station
L1 BTB	128-entry, 2-way, 1-cycle latency
L2 BTB	8192-entry, 4-way, 2-cycle latency
Branch direction predictor	TAGE-SC [23], 8-table, 1024-entry/table
Indirect predictor	ITTAGE [34], 8-table, 512-entry/table
Return address stack	32-entry
Branch predictor bandwidth	Up to 12-instruction or 1-taken per cycle
L1 Instruction Cache	32KB, 8-way, 16 MSHRs, 4-cycle
L1 Data Cache	48KB, 12-way, 16 MSHRs, 5-cycle, Next Line Prefetcher (NL)
L2 Unified Cache	512KB, 8-way, 32 MSHRs, 15-cycle, Signature Path Prefetcher (SPP) [41]
L3 Unified Cache	2MB, 16-way, 64 MSHRs, 35-cycle
DRAM	1 channel, 3200MT/s (25.6GB/s)

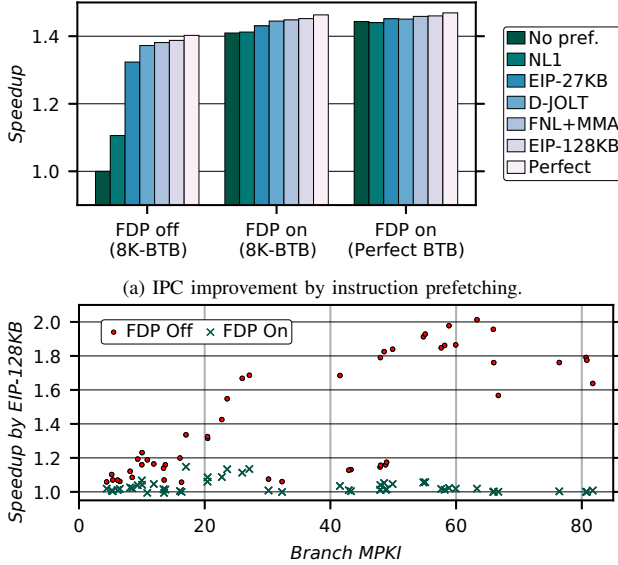


Fig. 6. Instruction prefetching performance results.

We report the geometric mean for instructions per cycle (IPC) and the arithmetic mean for mispredictions per kilo instructions (MPKI) using all evaluated workloads.

VI. RESULTS

A. Performance Improvement by FDP

We evaluate prefetching mechanisms with and without FDP. Fig. 6a shows speedup over the baseline (no prefetching, no FDP). Without FDP, NL1 and EIP-27KB offer 10.6% and 32.4% speedup, respectively. Three IPC-1 prefetchers,

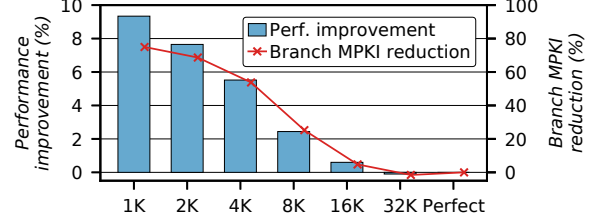


Fig. 7. Performance improvement and branch misprediction reduction achieved by enabling PFC for different BTB sizes.

D-JOLT, FNL+MMA, and EIP-128KB, show 37.2%, 38.1%, and 38.8% speedup, respectively. Their results are close to Perfect (40.3%), but FDP alone realizes a 41.0% speedup thanks to 1) instruction prefetches by FDP and 2) BTB access latency hidden by using FTQ (§IV-B). If the FTQ contains enough pending transactions, the pipeline can tolerate some bubbles caused by BTB access latency. FDP outperforms both EIP-27KB and EIP-128KB by 8.6% and 2.2%, respectively.

FDP shows significantly better performance over the baseline, and additional performance can be obtained in two ways: better branch target prediction and instruction prefetching. As explained in Section III-A, BTB misses limit the effectiveness of FDP. We identify that a perfect BTB improves the performance of FDP by 3.4%, which also shows the upper bound of BTB prefetching [10]–[13]. Better instruction prefetching shows slightly higher performance uplifts. FDP with EIP-128KB and Perfect give 4.3% and 5.4% additional performance improvements, respectively. When both perfect BTB and prefetching are used, it provides 46.9% performance improvement.

Fig. 6b shows performance improvements of EIP-128KB for each workload with FDP on and off. In this chart, each workload has two data points (performance improvements with/without FDP) that share the same x-coordinate as the branch MPKI remains unchanged. When FDP is disabled, EIP-128KB improves performance by up to 2.01x, and most workloads with greater than 20 MPKI show higher than 50% performance improvement. However, with FDP enabled, the max speedup falls to 14.8%, and two workloads show a 0.5% performance degradation. This result shows that FDP can cover most I-cache misses, while EIP-128KB helps capture some of the remaining hard-to-predict patterns.

B. Performance Impact of Post-Fetch Correction

As explained in Section III-B, PFC can improve the effectiveness of FDP, particularly with smaller BTBs. To confirm this, we evaluate PFC with BTB sizes varying from 1K to 32K entries by increasing the number of sets. Fig. 7 shows that PFC results in 2.4% and 9.3% better performance than when PFC is not enabled for an 8K-entry and an 1K-entry BTB, respectively. The performance benefit of PFC comes from reduced mispredictions (25.2% and 75.0% for an 8K-entry and an 1K-entry BTB, respectively).

However, PFC is less useful for larger BTBs, which is expected since PFC is proposed to handle BTB capacity misses.

TABLE V
BRANCH HISTORY MANAGEMENT POLICIES.

Configuration	Type	Not-taken branch history correction	BTB allocation
FDP (THR)	Target	Not necessary	Taken-only
Idealized GHR (Ideal)	Direction	Fix history w/o flush	Taken-only
GHR config 0 (GHR0)	Direction	Do not fix	Taken-only
GHR config 1 (GHR1)	Direction	Do not fix	All
GHR config 2 (GHR2)	Direction	Fix history and flush	Taken-only
GHR config 3 (GHR3)	Direction	Fix history and flush	All

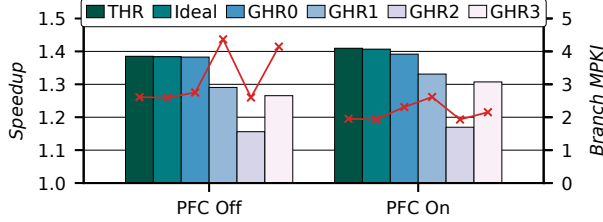


Fig. 8. Branch history management results (line: branch MPKI).

For strongly biased branches, PFC can be rather harmful. For example, any *never-taken* branch will not be allocated in the BTB while the large BTB has all other previously taken branches. When a direction predictor makes a wrong prediction (taken) for this BTB-miss branch, PFC incorrectly fixes the FTQ, re-streering the instruction stream to the wrong taken path and eventually leading to a pipeline flush. As a result, PFC increases mispredictions by 1.5% with only 0.1% performance uplift for a 32K-entry BTB.

C. Performance Impact of Branch History Management

Many modern processors use taken-only branch target history [37], but we notice that academic papers generally use other configurations (§III-A). To see the performance impact of history management schemes, we evaluate configurations with different history types, history corrections, and BTB allocation policies, as shown in Table V. Among GHR configurations evaluated, GHR3 with PFC off is commonly used with basic-block-based BTBs. Note that we selected 280-bit branch direction history from an empirical study using the idealized GHR configuration to achieve the best branch prediction accuracy.

Fig. 8 shows the performance and branch mispredictions per kilo instruction (MPKI) results. First of all, enabling PFC shows better performance across all configurations by reducing branch misprediction. Since the overall trend is similar between PFC on and PFC off, we will describe results with PFC enabled now on. THR shows similar performance to Ideal, showing the benefit of using branch target history. Since we tuned the branch predictors to improve accuracy for each configuration, THR in some cases achieves a marginally higher accuracy than Ideal. Compared to Ideal, GHR2 shows similar branch prediction accuracy, but frontend flushes to fix up the history incur more pipeline stalls, resulting in

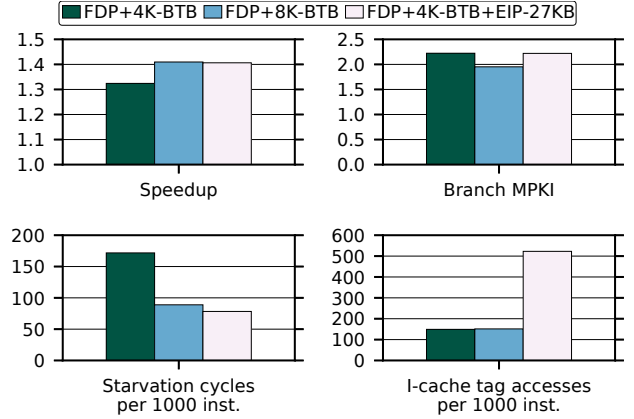


Fig. 9. ISO-budget analysis.

23.7% lower performance. Not fixing the history as in GHR0 increases branch mispredictions by 19.5% with 1.5% lower performance than Ideal. The taken-only BTB allocation policies show better prediction accuracy (GHR0 vs. GHR1 and GHR2 vs. GHR3), but the performance impact varies depending on whether we correct not-taken branch histories. Both GHR2 and GHR3 require pipeline flushes for BTB-miss not-taken branches to fix history. GHR3 incurs more mispredictions as it allocates entries for not-taken branches as well, but this helps decrease the pipeline flushes.

This experiment shows that the FDP configuration with taken-only branch target history (THR) outperforms all other configurations, and, thus, the target history should be used in the frontend design.

D. ISO-Budget Comparison

Dedicated instruction prefetchers offer some performance improvement on top of FDP (§VI-A), but these prefetching mechanisms require additional storage and complex logic. Based on Samsung Exynos M3 data [27], [28], we estimated that a BTB entry needs 7 bytes per branch. Since EIP-27KB (§V) has similar storage cost as a 4K-entry BTB, we compare 1) 8K-entry BTB, 2) 4K-entry BTB with EIP-27KB, and 3) 4K-entry BTB as a reference, on top of FDP.

Fig. 9 shows speedup, branch MPKI, starvation cycles per kilo instructions, and the number of I-cache tag accesses per kilo instructions. Here, *starvation* is due to fetch-stalls, and *starvation cycles* is defined as the total number of cycles when the decode queue contains less than a decode-width number of instructions. Both configurations perform similarly (41.0% vs. 40.6% speedup), although the two mechanisms take different approaches. We find that thanks to a larger BTB, FDP with an 8K-BTB has 12% fewer branch mispredictions compared to with EIP-27KB. However, useful prefetches by EIP-27KB result in 13.5% lower starvation cycles compared to FDP with an 8K-BTB. The biggest difference between the two schemes is the number of I-cache tag accesses. Prefetching mechanisms inevitably have redundant prefetches, which are less harmful than inaccurate prefetches. However,

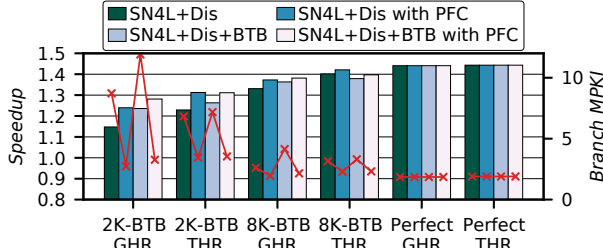


Fig. 10. BTB prefetching results (line: branch MPKI).

without an effective filtering mechanism, redundant prefetches will probe the I-cache tag array.³ As a result, as shown in Fig. 9, EIP-27KB has 3.5 times more I-cache tag accesses, which will increase dynamic cache power consumption. This study suggests that the complexity and power consumption by having an additional prefetching mechanism cannot be justified over the straightforward design of FDP with a larger BTB.

E. BTB Prefetching with PFC

As explained in Section II-B, BTB is critical to FDP effectiveness. BTB prefetching [10]–[13] has been proposed to compensate for limited BTB capacities. When a BTB miss occurs, a branch will go undetected and be mispredicted if the branch is taken. Unlike instruction prefetching mechanisms, BTB prefetching requires an additional pre-decoder to identify branch instructions from the I-cache lines. Note that our FDP design also requires a pre-decoder for PFC.

In this section, we evaluate Divide-and-conquer [13], which is based on FDP and comprises three predictors: selective-next-four-line (SN4L), discontinuity [4] (Dis), and BTB prefetching. SN4L improves next-four-line prefetcher using usefulness filter. Only previously useful cache lines among the next four will be prefetched. Dis records jumps between two I-cache miss lines in DisTable. On I-cache accesses, Dis looks up DisTable and generates prefetches if the missing line is found. Otherwise, the BTB will be consulted. Finally, BTB prefetching is employed to reduce BTB misses. BTB prefetching can be activated when an I-cache fill is processed. The pre-decoder identifies the PC-relative branches and installs those detected branches unconditionally into the BTB array. Because of the nature of the prefetching scheme, the register-relative indirect branches cannot be prefetched by this mechanism.

We evaluate Divide-and-conquer with BTB prefetching (SN4L+Dis+BTB) and without BTB prefetching (SN4L+Dis) on different BTB sizes (2K-, 8K-entry, and perfect BTBs), different branch history managements (THR: target history in our FDP design and GHR: GHR3 in Table V used in academic papers), and with/without PFC. Fig. 10 shows the result, and we make the following observations. PFC is more effective than BTB prefetching. In the case of a BTB miss, BTB prefetching causes pipeline stalls until the BTB miss is resolved. However, PFC can speculatively progress without

³FNL+MMA [31] implemented such a filter, but it still requires 1.5x more I-cache tag accesses over no prefetch.

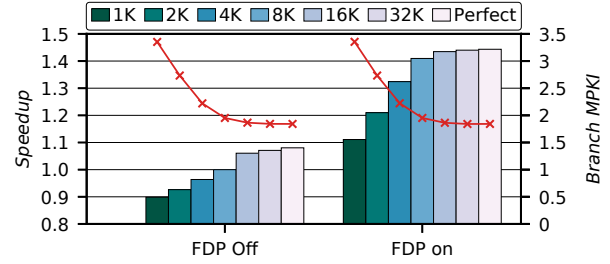


Fig. 11. BTB capacity sensitivity (line: branch MPKI).

waiting for BTB miss resolution, as long as the BTB miss branch satisfies a PFC condition. As we demonstrated in Section VI-C, we can see that THR is always more beneficial than GHR. Also, we can observe that BTB prefetching is less effective with larger BTBs. When GHR is used, BTB prefetching improves performance by 8.8% and 3.2% for 2K- and 8K-entry BTBs, respectively. For 8K-entry BTB with THR, BTB prefetching hurts performance. The performance degradation is due to BTB pollution caused by prefetching strongly biased, e.g., (almost) never taken, branches into the BTB. These branches victimize useful BTB entries, including non-prefetchable indirect branches. This result shows the limitation of BTB prefetching for commercial processors that employ larger BTBs.

F. Branch Prediction Sensitivity

1) *BTB capacity*: As explained in Section I, FDP requires a sufficiently large BTB capacity to be effective. To see how FDP works with different BTB sizes, we vary the size from 1K to 32K entries. Fig. 11 shows the result. With FDP, PFC can compensate for more BTB misses in smaller BTBs, resulting in much higher performance improvements in smaller BTBs (between 1K- to 8K-entry). However, without FDP, we can see only moderate performance improvement by increasing BTB size and the largest jump at 16K-entry BTB, i.e., branch footprint fits in 16K-entry BTB. After 16K entries, both configurations show diminishing performance benefits. As explained in Section IV-C, although the number of branch mispredictions is similar, FDP performs significantly better in all BTB capacities because BTB and I-cache access latencies can be well hidden by FDP.

2) *Branch direction predictor sensitivity*: FDP also relies on accurate branch direction prediction. In this section, we evaluate Gshare [21] (8KB) with a 15-bit idealized branch direction history, various TAGE [23] sizes, including 9KB (a half), 18KB (baseline), 36KB (2x), and perfect prediction. For the TAGE predictor, we change the number of entries in all prediction tables without optimizing the history length.

Fig. 12 shows the result. As expected, FDP with the Gshare predictor performs worse than a similarly-sized TAGE (31.4% vs. 37.1%). Also, PFC degrades performance with Gshare, unlike with TAGE predictors, resulting in 6.0% performance loss. With less accurate direction predictors, BTB-miss not-taken branches with an incorrectly predicted direction (predicted

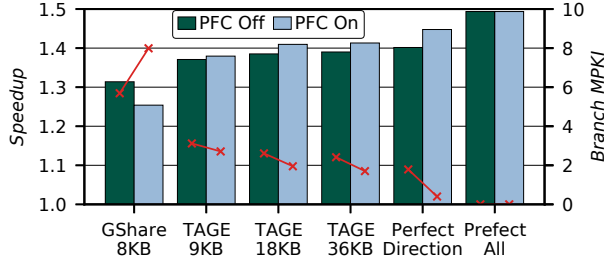


Fig. 12. Branch predictor sensitivity (line: branch MPKI).

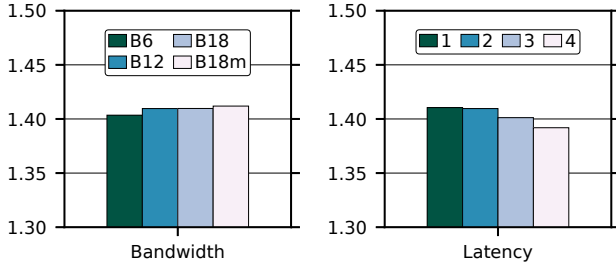


Fig. 13. BTB latency/bandwidth sensitivity results (y-axis: speedup).

taken) are correctly handled without PFC. However, PFC incorrectly fixes the branch prediction, causing a misprediction and eventually a pipeline flush at branch resolution time. This result demonstrates that conventional wisdom that says “do not make taken prediction upon a BTB miss” has changed thanks to the improvement in branch direction prediction algorithms. Using perfect direction prediction makes PFC more effective (4.6% improvement), and using both perfect direction and target predictions (Perfect All) leads to 49.4% performance improvement.

3) *Prediction bandwidth/latency sensitivity*: We also vary prediction bandwidth and BTB latency parameters. Fig. 13 shows the results. As explained in Section V, we use 2x higher prediction bandwidth (12-instruction per cycle) than fetch bandwidth (6-instruction per cycle) to improve run-ahead capability. Also, we stop the prediction after a predicted-taken branch is found. In this section, we evaluate 6-instruction per cycle (a half bandwidth, B6), 12 (the baseline, B12), 18 (1.5x, B18), and 18 with multiple taken branches (B18m) configurations. B18 does not improve performance over B12 since 2x fetch bandwidth can sufficiently feed instructions to the decode stage. Having half the bandwidth degrades performance by 0.6%. When we predict more than one taken branch, it (B18m) gives a marginal 0.2% performance improvement.

We also vary the BTB latency from one to four cycles. In a steady state, the BTB latency is less critical since BTB accesses are pipelined, but the latency affects performance when the FTQ does not contain enough instructions after pipeline flushes. Compared to the baseline 2-cycle latency, 4-cycle latency hurts performance by 1.8%.

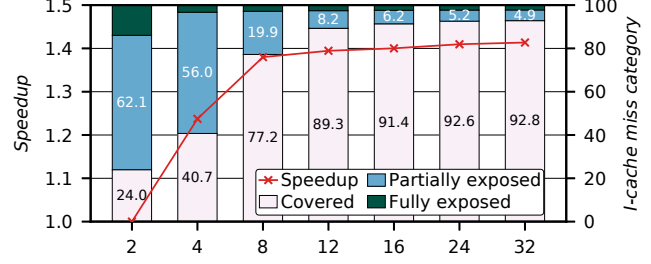


Fig. 14. FTQ size sensitivity (x-axis: FTQ size).

G. FTQ Size Sensitivity

We evaluate the performance sensitivity of FDP to the FTQ size, sweeping it from 2 to 32 entries. Fig. 14 shows the performance normalized to a 2-entry FTQ (which is equivalent to no FDP). The speedup increases from 23.7% for a 4-entry FTQ to 39.5% for a 12-entry FTQ, and the increase is marginal thereafter.

To further understand performance gains from larger FTQ sizes, we classify cache misses into 1) fully exposed, 2) partially exposed, or 3) covered. A cache miss is *covered* if the corresponding cache transaction receives the cache line before observing a starvation cycle. A cache miss is *fully exposed* only if a cache miss request is initiated after the corresponding FTQ entry becomes the head of FTQ, otherwise *partially exposed*. As shown in Fig. 14, 76% of cache misses are fully or partially exposed with a 2-entry FTQ, and FDP with a 24-entry FTQ removes 90.6% of such exposed cache misses. The exposed misses are continuously reduced up to 32-entry, mostly from partially exposed misses. Consequently, reducing fully exposed cache misses could be one focus area for a dedicated prefetching algorithm.

VII. RELATED WORK

Instruction and BTB prefetching are mutually complementary. BTB prefetching requires to see instruction streams to decode branch information from the instruction block and fill the information into the BTB. For instruction prefetching to be effective, it requires accurate prediction of branch target information from the BTB. In this section, we will discuss these two prefetching mechanisms.

A. Instruction Prefetching

Unlike spatial prefetchers, such as next line prefetching [19], [20], that prefetch adjacent I-cache lines, temporal prefetching [5], [6], [18] can track temporally correlated streams of I-cache misses and prefetch subsequent addresses upon seeing triggering lines again. Spatial compaction is often applied to temporal streams [6]. SHIFT [7] reduced the storage overhead of temporal history by sharing it across cores. Call graph prefetching (CGP) [42] analyzed call graphs to prefetch instructions of the function that may soon be called. RAS-directed instruction prefetching (RDIP) [9] correlated I-cache misses with the program context captured from the RAS.

When a program encounters the same context, recorded I-cache missing lines will be prefetched. The main drawback of temporal prefetching mechanisms is that they require large metadata storage to record the instruction streams to be effective.

Various hardware instruction prefetching mechanisms were published in the IPC-1 [14]. EIP [18] introduced the concept of entangling the cache line to be prefetched (destination) with a source cache line such that the destination would be prefetched when the source cache line is encountered. FNL+MMA [31] combines efficient, aggressive next line prefetcher (Footprint Next Line) and temporal correlation prefetcher (Multiple Miss Ahead). D-JOLT [30] improves upon RDIP by using a new signature generation mechanism that generates the prefetches from a FIFO structure that stores the function return addresses instead of the stack structure used by RDIP. Most of these recent prefetching mechanisms require large structures and complex logic, while we show that FDP can achieve similar benefits by leveraging existing branch prediction structures.

Software prefetching mechanisms also exist [15]–[17], [43]. pTask [16] identified that operating system (OS)-intensive applications keep switching tasks between applications, system calls, and interrupt handlers, and the I-cache hit rate drops significantly right after task switches. A specific OS event generally separated these tasks. pTask annotates the OS and applications to profile frequent functions, associates them with OS events, and records the compressed prefetch list in memory, which is later used to issue prefetches. However, pTask was evaluated without a realistic BTB implementation and did not faithfully model a decoupled frontend. AsmDB [15] and I-SPY [17] are recent profile-guided compiler-based software instruction prefetching proposals. While these approaches may be effective, they were evaluated using a simple frontend design that lacked FDP and realistic branch prediction structures, such as BTB and RAS.

B. BTB Prefetching

Confluence [10] proposed unified metadata for instruction and BTB prefetching. The metadata includes a branch type and a target displacement field for an I-cache line. To incorporate per-block metadata, they also proposed a new block-based BTB construction, AirBTB, which has bitmaps to identify branches in a block and stores other metadata related to the branch (type and target information). Boomerang [11] was proposed to remove metadata needed by Confluence by using a basic-block-based BTB implementation. Upon BTB misses, the corresponding I-cache line will be fetched and pre-decoded to identify branches in it. The identified branches will be filled in the BTB. However, more complex basic-block-based BTBs require additional storage to store additional information, such as basic-block size, and track (almost) always not-taken branches. They also need to be accessed to identify all branches, including not-taken branches, to decide the start address of the next basic-block, limiting the BTB bandwidth while being more power-hungry. Shotgun [12] optimizes the BTB construction with three BTB structures: unconditional

branch BTB (U-BTB), return instruction buffer (RIB), and conditional branch BTB (C-BTB). U-BTB incorporates the spatial footprint of calls and long jumps. For these remote branches, U-BTB stores spatial information at both caller and callee sites. Upon function calls and returns, prefetches will be generated based on their spatial coding.

VIII. CONCLUSIONS

Addressing frontend performance bottlenecks is critical for applications with large code footprints. However, academic and industrial communities have diverged and are using different approaches to resolve the same issue. We revisited FDP that many commercial CPUs have employed, presented a comprehensive design, and evaluated two features, taken-only branch target history and post-fetch correction, to address challenges in designing an effective FDP. Our FDP design offers 41.0% speedup over no prefetching with marginal 195 bytes of hardware overhead. It also outperforms IPC-1 winners and performs 5.4% lower than perfect prefetching. This design can also be coupled with other prefetchers to further improve performance for more frontend intensive workloads. We believe that our work can guide future instruction prefetching research to be more impactful to commercial CPUs.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their valuable feedback.

REFERENCES

- [1] I-Cheng K. Chen, Chih-Chieh Lee, and T. N. Mudge, "Instruction prefetching using branch prediction information," in *Proceedings of the International Conference on Computer Design: VLSI in Computers and Processors (ICCD)*, 1997, pp. 593–601.
- [2] A. V. Veidenbaum, "Instruction cache prefetching using multilevel branch prediction," in *Proceedings of the International Symposium on High Performance Computing (ISHPC)*. Springer, 1997, pp. 51–70.
- [3] V. Srinivasan, E. S. Davidson, G. S. Tyson, M. J. Charney, and T. R. Puzak, "Branch history guided instruction prefetching," in *Proceedings of the 7th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2001, pp. 291–300.
- [4] L. Spracklen, Yuan Chou, and S. G. Abraham, "Effective instruction prefetching in chip multiprocessors for modern commercial applications," in *Proceedings of the 11th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2005, pp. 225–236.
- [5] M. Ferdman, T. F. Wenisch, A. Ailamaki, B. Falsafi, and A. Moshovos, "Temporal instruction fetch streaming," in *Proceedings of the 41st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. USA: IEEE Computer Society, 2008, pp. 1–10. [Online]. Available: <https://doi.org/10.1109/MICRO.2008.4771774>
- [6] M. Ferdman, C. Kaynak, and B. Falsafi, "Proactive instruction fetch," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. New York, NY, USA: Association for Computing Machinery, 2011, pp. 152–162. [Online]. Available: <https://doi.org/10.1145/2155620.2155638>
- [7] C. Kaynak, B. Grot, and B. Falsafi, "SHIFT: Shared history instruction fetch for lean-core server processors," in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2013, pp. 272–283.
- [8] G. Reinman, B. Calder, and T. Austin, "Fetch directed instruction prefetching," in *Proceedings of the 32nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. USA: IEEE Computer Society, 1999, pp. 16–27.

- [9] A. Kolli, A. Saidi, and T. F. Wenisch, "RDIP: Return-address-stack directed instruction prefetching," in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. New York, NY, USA: Association for Computing Machinery, 2013, pp. 260–271. [Online]. Available: <https://doi.org/10.1145/2540708.2540731>
- [10] C. Kaynak, B. Grot, and B. Falsafi, "Confluence: Unified instruction supply for scale-out servers," in *Proceedings of the 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. New York, NY, USA: Association for Computing Machinery, 2015, pp. 166–177. [Online]. Available: <https://doi.org/10.1145/2830772.2830785>
- [11] R. Kumar, C. Huang, B. Grot, and V. Nagarajan, "Boomerang: A metadata-free architecture for control flow delivery," in *Proceedings of the 23rd IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2017, pp. 493–504.
- [12] R. Kumar, B. Grot, and V. Nagarajan, "Blasting through the front-end bottleneck with shotgun," in *Proceedings of the 23rd ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. New York, NY, USA: Association for Computing Machinery, 2018, pp. 30–42. [Online]. Available: <https://doi.org/10.1145/3173162.3173178>
- [13] A. Ansari, P. Lotfi-Kamran, and H. Sarbazi-Azad, "Divide and conquer frontend bottleneck," in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 65–78.
- [14] IPC-1, "The 1st instruction prefetching championship," <https://research.ece.ncsu.edu/ipc/>, 2020.
- [15] G. Ayers, N. P. Nagendra, D. I. August, H. K. Cho, S. Kanev, C. Kozyrakis, T. Krishnamurthy, H. Litz, T. Moseley, and P. Ranganathan, "AsmDB: Understanding and mitigating front-end stalls in warehouse-scale computers," in *Proceedings of the 46th Annual International Symposium on Computer Architecture (ISCA)*. New York, NY, USA: Association for Computing Machinery, 2019, pp. 462–473. [Online]. Available: <https://doi.org/10.1145/3307650.3322234>
- [16] P. Kallurkar and S. R. Sarangi, "pTask: A smart prefetching scheme for os intensive applications," in *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–12.
- [17] T. A. Khan, A. Sriraman, J. Devietti, G. Pokam, H. Litz, and B. Kasikci, "I-SPY: Context-driven conditional instruction prefetching with coalescing," in *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 146–159.
- [18] A. Ros and A. Jimborean, "The entangling instruction prefetcher," *IEEE Computer Architecture Letters (CAL)*, vol. 19, no. 2, pp. 84–87, 2020.
- [19] J. E. Smith and W.-C. Hsu, "Prefetching in supercomputer instruction caches," in *Proceedings of the 1992 ACM/IEEE Conference on Supercomputing (SC)*. Washington, DC, USA: IEEE Computer Society Press, 1992, pp. 588–597.
- [20] G.-H. Park, O.-Y. Kwon, T.-D. Han, S.-D. Kim, and S.-B. Yang, "An improved lookahead instruction prefetching," in *Proceedings High Performance Computing on the Information Superhighway (HPC Asia)*, 1997, pp. 712–715.
- [21] S. McFarling, "Combining branch predictors," in *Technical Report TN-36, Digital Western Research Laboratory*, June 1993.
- [22] D. A. Jimenez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proceedings of the 7th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. Washington, DC, USA: IEEE Computer Society, Jan 2001, pp. 197–206.
- [23] A. Seznec, "TAGE-SC-L branch predictors again," *The 5th JILP Workshop on Computer Architecture Competitions (JWAC-5): Championship Branch Prediction (CBP-5)*, 2014.
- [24] G. Reinman, T. Austin, and B. Calder, "A scalable front-end architecture for fast instruction delivery," in *Proceedings of the 26th Annual International Symposium on Computer Architecture (ISCA)*. USA: IEEE Computer Society, 1999, pp. 234–245. [Online]. Available: <https://doi.org/10.1145/300979.300999>
- [25] N. Adiga, J. Bonanno, A. Collura, M. Heizmann, B. R. Prasky, and A. Saporito, "The IBM z15 High Frequency Mainframe Branch Predictor," in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 27–39.
- [26] A. Pellegrini, N. Stephens, M. Bruce, Y. Ishii, J. Pusdesris, A. Raja, C. Abernathy, J. Koppanalil, T. Ringe, A. Tummala, J. Jalal, M. Werkheiser, and A. Kona, "The Arm Neoverse N1 platform: Building blocks for the next-gen cloud-to-edge infrastructure SoC," *IEEE Micro*, vol. 40, no. 2, pp. 53–62, 2020.
- [27] J. Rupley, "Samsung Exynos M3 processor," in *2018 IEEE Hot Chips 30 Symposium (HCS)*, 2018.
- [28] B. Grayson, J. Rupley, G. Z. Zuraski, E. Quinell, D. A. Jiménez, T. Nakra, P. Kitchin, R. Hensley, E. Brekelbaum, V. Sinha, and A. Ghiya, "Evolution of the Samsung Exynos CPU Microarchitecture," in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 40–51.
- [29] D. Suggs, M. Subramony, and D. Bouvier, "The AMD "Zen 2" processor," *IEEE Micro*, vol. 40, no. 2, pp. 45–52, 2020.
- [30] T. Nakamura, T. Koizumi, Y. Degawa, H. Irie, S. Sakai, and R. Shioya, "D-JOLT: Distant jolt prefetcher again," *The 1st Instruction Prefetching Championship (IPC-1)*, 2020.
- [31] A. Seznec, "The FNL+MML instruction cache prefetcher," *The 1st Instruction Prefetching Championship (IPC-1)*, 2020.
- [32] P. Michaud, "PIPS: Prefetching instructions with probabilistic scouts," *The 1st Instruction Prefetching Championship (IPC-1)*, 2020.
- [33] T.-Y. Yeh and Y. N. Patt, "Two-level adaptive training branch prediction," in *Proceedings of the 24th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. New York, NY, USA: ACM, 1991, pp. 51–61. [Online]. Available: <http://doi.acm.org/10.1145/123465.123475>
- [34] A. Seznec, "A 64-kbytes ITTAGE indirect branch predictor," *The 2nd JILP Workshop on Computer Architecture Competitions (JWAC-2): Championship Branch Prediction (CBP-3)*, 2011.
- [35] A. Seznec, S. Felix, V. Krishnan, and Y. Sazeides, "Design tradeoffs for the Alpha EV8 conditional branch predictor," in *Proceedings of the 29th Annual International Symposium on Computer Architecture (ISCA)*. Washington, DC, USA: IEEE Computer Society, 2002, pp. 295–306.
- [36] V. Uzelac and A. Milenkovic, "Experiment flows and microbenchmarks for reverse engineering of branch predictor structures," in *Proceedings of the 2009 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2009, pp. 207–217.
- [37] AMD, "Software optimization guide for amd family 17h processors," https://developer.amd.com/wordpress/media/2013/12/55723_SOG_Fam_17h_Processors_3.00.pdf, 2017.
- [38] "ChampSim," <https://github.com/ChampSim/ChampSim>.
- [39] Standard Performance Evaluation Corporation, "SPEC CPU benchmark suites," <https://www.spec.org/cpu/>, 2017.
- [40] I. Cutress, "Examining Intel's Ice Lake processors: Taking a bite of the Sunny Cove microarchitecture," <https://www.anandtech.com/show/14514/examining-intels-ice-lake-microarchitecture-and-sunny-cove/3>, 2019.
- [41] J. Kim, S. H. Pugsley, P. V. Gratz, A. L. N. Reddy, C. Wilkerson, and Z. Chishti, "Path confidence based lookahead prefetching," in *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–12.
- [42] M. Annavaram, J. M. Patel, and E. S. Davidson, "Call graph prefetching for database applications," in *Proceedings of the 7th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2001, pp. 281–290.
- [43] Chi-Keung Luk and T. C. Mowry, "Cooperative prefetching: compiler and hardware support for effective instruction prefetching in modern processors," in *Proceedings of the 31st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 1998, pp. 182–193.